Installer l’Atelier avec Docker

Guide pas à pas — votre ordinateur ou un serveur (VPS)

Octobre 2026 · branche `docker`

Sommaire

En bref 1

1. Ce qu’il vous faut 2

2. Installer Docker 2

3. Récupérer le code 2

4. Construire l’image 3

5. Choisir le mot de passe du tableau de bord 3

6. Démarrer 3

7. Ouvrir le tableau de bord 3

8. Connecter les assistants 4

9. Vérifier que tout fonctionne 4

10. Au quotidien 5

11. Dépannage 6

12. Sécurité, en bref 7

En bref

L’Atelier tourne dans un **conteneur Docker** : une petite machine isolée qui contient le tableau de bord, LibreOffice, Pandoc, XeLaTeX, Claude Code et Codex. Vos données et les connexions des assistants sont rangées dans des **volumes** : elles survivent aux redémarrages et aux mises à jour.

Les commandes, dans l’ordre :

```
git clone https://github.com/Maxime-Skryma/atelier-agents.git
cd atelier-agents
git switch docker
docker compose build
docker compose run --rm atelier init
docker compose up -d
docker exec -it atelier claude
docker exec -it atelier codex login --device-auth
```

Puis ouvrez http://localhost:8765/. Le détail de chaque étape suit.

> **Toutes les commandes se tapent dans le Terminal de votre ordinateur** (sur Mac : application *Terminal*), **dans le dossier `atelier-agents`**, sauf mention contraire.

1. Ce qu'il vous faut

- **Docker Desktop** (Mac ou Windows) ou **Docker Engine** avec le module *Compose* (Linux, VPS).
- **Environ 10 Go d'espace disque** : l'image fait plusieurs Go, surtout à cause de LibreOffice et de XeLaTeX.
- **8 Go de mémoire réservés à Docker**, conseillés : le conteneur est limité à 6 Go.
- **Git**, pour récupérer le code. Sur Mac, il s'installe à la première commande `git` (outils de développement d'Apple).
- **Au moins un abonnement** : Claude (Pro ou Max), ChatGPT (Plus ou Pro) ou Google (pour Antigravity).

2. Installer Docker

Sur Mac. Téléchargez *Docker Desktop* sur https://www.docker.com/products/docker-desktop/, en choisissant la version *Apple Silicon* (puces M1 à M4) ou *Intel* selon votre Mac (menu Pomme › *À propos de ce Mac*). Ouvrez le fichier `.dmg`, glissez Docker dans *Applications*, puis lancez-le une fois et acceptez les conditions.

Réglez ensuite la mémoire : Docker Desktop › *Settings* › *Resources* › *Memory* : **8 Go** au moins, puis *Apply & restart*.

Sur un serveur Linux (Debian, Ubuntu). Suivez la documentation officielle *Install Docker Engine* pour votre distribution (https://docs.docker.com/engine/install/), qui installe aussi le module Compose.

Vérifiez dans le Terminal :

```
docker --version
docker compose version
```

Les deux commandes doivent afficher un numéro de version. Si `docker compose` est inconnu, votre Docker est trop ancien : mettez-le à jour.

3. Récupérer le code

```
cd ~
git clone https://github.com/Maxime-Skryma/atelier-agents.git
cd atelier-agents
git switch docker
```

La dernière ligne place le dossier sur la branche **`docker`**, qui contient l'installation Docker. Une fois cette branche fusionnée dans `main`, elle ne sera plus nécessaire.

4. Construire l'image

```
docker compose build
```

La première fois, comptez **10 à 20 minutes** selon votre connexion : Docker télécharge Python, Node, LibreOffice, XeLaTeX, Claude Code et Codex. Les fois suivantes, seules les parties modifiées sont refaites.

Options, à ajouter si besoin :

Option	Effet
`--build-arg WITH_LATEX=0`	image plus légère, sans XeLaTeX : pas de livrables PDF
`--build-arg CODEX_VERSION=…`	fixe la version de Codex (par défaut : la dernière)
`--build-arg CLAUDE_CODE_VERSION=…`	fixe la version de Claude Code (par défaut : la dernière)

5. Choisir le mot de passe du tableau de bord

```
docker compose run --rm atelier init
```

Tapez un mot de passe de **12 caractères au moins**, puis confirmez-le. Rien ne s'affiche pendant la saisie : c'est normal. Le mot de passe est **haché** (illisible) dans le volume de données, jamais écrit en clair.

> **Autre possibilité.** Créez un fichier `.env` à côté de `docker-compose.yml`, contenant `ATELIER_PASSWORD=votre-mot-de-passe`. Il sera utilisé au premier démarrage seulement. **Supprimez ensuite cette ligne** : le fichier `.env` reste en clair sur votre disque.

6. Démarrer

```
docker compose up -d
```

Le conteneur `atelier` démarre en arrière-plan et **redémarre tout seul** avec Docker, sauf si vous l'avez arrêté vous-même. Vérifiez qu'il tourne :

```
docker compose ps
```

La colonne *STATUS* doit afficher `Up … (healthy)` au bout de 30 secondes environ.

7. Ouvrir le tableau de bord

Ouvrez **http://localhost:8765/** dans votre navigateur, puis connectez-vous avec le mot de passe de l'étape 5.

Le tableau de bord n'est accessible **que depuis la machine elle-même** : le port est publié sur `127.0.0.1` uniquement.

Sur un serveur distant (VPS), ouvrez un tunnel SSH depuis votre ordinateur :

```
ssh -L 8765:127.0.0.1:8765 utilisateur@adresse-du-serveur
```

Laissez cette fenêtre ouverte, puis ouvrez http://localhost:8765/ sur votre ordinateur. Derrière un proxy HTTPS, indiquez plutôt l'adresse publique dans `.env` (`ATELIER_PUBLIC_URL=https://votre-adresse/`), puis relancez avec `docker compose up -d`.

8. Connecter les assistants

Les assistants utilisent **vos abonnements**, jamais de clé d'API. Les sessions sont rangées dans des volumes : elles restent après un redémarrage ou une mise à jour.

Claude Code

```
docker exec -it atelier claude
```

Tapez `/login`, choisissez votre abonnement Claude, ouvrez le lien affiché dans votre navigateur, puis recollez le code obtenu dans le Terminal. Quittez avec `/exit`.

Codex (ChatGPT)

```
docker exec -it atelier codex login --device-auth
```

Ouvrez le lien affiché, connectez-vous à ChatGPT et saisissez le code indiqué dans le Terminal. Gardez bien l'option `--device-auth` : sans elle, Codex attend une réponse sur le `localhost` du conteneur, que votre navigateur ne peut pas joindre.

Gemini · Antigravity (facultatif)

`agy` n'est pas inclus dans l'image. Installez-le **dans le conteneur** en suivant la documentation de Google (ouvrez un terminal avec `docker exec -it atelier bash`). Placez son exécutable dans `/opt/agy/bin/`, qui est un volume conservé, puis connectez-vous :

```
docker exec -it atelier agy
```

Vérifiez enfin dans le tableau de bord : *Connexions & réglages › Vérifier les connexions*.

9. Vérifier que tout fonctionne

Cochez chaque point.

☐ **Le tableau de bord répond et la connexion fonctionne**

Étape 7 : la page s'ouvre, un mauvais mot de passe est refusé, le bon est accepté.

☐ **Une mission de démonstration va jusqu'au livrable**

Dans le tableau de bord : *Nouvelle mission*, choisissez une équipe, cochez **« Essayer en démonstration »** et lancez. La démonstration ne fait appel à aucun assistant et ne consomme aucun quota. La mission doit se terminer et proposer un livrable à télécharger.

☐ **PDF, PowerPoint et aperçus**

```
docker exec atelier bash /app/scripts/docker-check.sh
docker cp atelier:/tmp/atelier-test ./atelier-test
```

La première commande doit afficher `PDF: OK · PPTX: 2 slides`, puis `Previews: OK`. Ouvrez ensuite le dossier `atelier-test` : `test.pdf`, `test.pptx` et les images `pdf-1.png`, `pptx-1.png` et `pptx-2.png` sont des aperçus produits par LibreOffice et Poppler. Supprimez ce dossier une fois vérifié.

☐ **Les données et les connexions persistent**

```
docker compose down
docker compose up -d
```

Reconnectez-vous au tableau de bord : la mission de démonstration est toujours là, et *Vérifier les connexions* indique toujours Claude et Codex connectés.

> **N'ajoutez jamais `-v`** à `docker compose down` : `docker compose down -v` **efface les volumes**, c'est-à-dire vos données et vos connexions.

☐ **Les tests automatiques passent**

```
docker compose run --rm atelier python3 -m unittest discover -s tests
```

La dernière ligne doit être `OK`.

☐ **Une vraie mission**

Lancez une courte mission **réelle** avec un agent Claude et un agent Codex, la création de fichiers étant autorisée. Elle confirme que les abonnements fonctionnent depuis le conteneur.

10. Au quotidien

Pour…	Commande
voir l'état	`docker compose ps`
lire le journal	`docker compose logs -f` (Ctrl + C pour quitter)
arrêter	`docker compose stop`
redémarrer	`docker compose start`
ouvrir un terminal dans le conteneur	`docker exec -it atelier bash`
mettre à jour	`git pull && docker compose up -d --build`

La mise à jour reconstruit l'image et redémarre le conteneur. Vos données et vos connexions sont conservées. Évitez de la lancer pendant qu'une mission tourne : elle serait interrompue.

Sauvegarder les données

Les données sont dans le volume `atelier-agents_atelier-data` (Docker préfixe le nom avec celui du dossier). Pour en faire une archive dans le dossier courant :

```
docker compose stop
docker run --rm -v atelier-agents_atelier-data:/data -v "$PWD":/backup \
  debian:bookworm-slim tar czf /backup/atelier-data-$(date +%F).tar.gz -C /data .
docker compose start
```

L'arrêt garantit une base de données cohérente. L'archive contient le hachage du mot de passe : gardez-la en lieu sûr.

Restaurer une sauvegarde

```
docker compose stop
docker run --rm -v atelier-agents_atelier-data:/data -v "$PWD":/backup \
  debian:bookworm-slim sh -c \
  'rm -rf /data/* && tar xzf /backup/atelier-data-AAAA-MM-JJ.tar.gz -C /data'
docker compose start
```

Remplacez `AAAA-MM-JJ` par la date de l'archive. Cette opération **remplace** les données actuelles.

Où sont rangées les choses

Volume	Dans le conteneur	Contenu
`atelier-data`	`/data`	base, missions, documents, mot de passe haché, journaux
`claude`	`/home/atelier/.claude`	session Claude Code
`codex`	`/home/atelier/.codex`	session Codex
`gemini`	`/home/atelier/.gemini`	session Antigravity
`agy`	`/opt/agy`	programme agy

Gardez ces volumes sur le disque local de la machine, jamais sur un partage réseau : la base SQLite ne le supporte pas.

11. Dépannage

Problème	Que faire
`Cannot connect to the Docker daemon`	Docker Desktop n'est pas lancé : ouvrez-le et attendez la baleine fixe dans la barre des menus.
`port is already allocated`	Le port 8765 est pris, par exemple par une autre installation de l'Atelier. Arrêtez-la, ou remplacez `127.0.0.1:8765:8765` par `127.0.0.1:8766:8765` dans `docker-compose.yml` et ouvrez http://localhost:8766/.
Le conteneur redémarre en boucle, `No dashboard password yet`	Le mot de passe n'est pas défini : `docker compose run --rm atelier init`, puis `docker compose up -d`.
« Adresse non configurée »	Vous passez par une adresse autre que `localhost` : renseignez `ATELIER_PUBLIC_URL` dans `.env`, puis `docker compose up -d`.
La construction échoue au téléchargement	Problème réseau passager : relancez `docker compose build`.
Un assistant apparaît « déconnecté »	Refaites sa connexion (étape 8).

Problème	Que faire
Le PDF échoue	L'image a été construite avec `WITH_LATEX=0` : reconstruisez sans cette option.
Le conteneur s'arrête, mémoire saturée	Augmentez la mémoire de Docker Desktop, ou ajustez `mem_limit` dans `docker-compose.yml`.

Pour tout autre problème, lisez le journal avec `docker compose logs --tail 50`.

12. Sécurité, en bref

- **Le conteneur est la limite d'isolation.** Codex y tourne sans son propre bac à sable (`danger-full-access`). Il peut lire et modifier tout ce que l'utilisateur `atelier` atteint dans le conteneur, y compris les données et les sessions. Il n'a accès ni au code de l'application, qui est en lecture seule, ni aux fichiers de votre ordinateur.
- **Aucun dossier de votre ordinateur n'est monté**, ni le socket Docker. Le conteneur tourne sans droits administrateur, sans privilèges supplémentaires, avec des limites de mémoire, de processeur et de processus.
- **Moins étanche qu'une machine virtuelle** : le conteneur partage le noyau de votre système, et une faille du noyau pourrait permettre d'en sortir. Les agents ont aussi un accès complet à Internet.
- **Le tableau de bord n'écoute que sur votre machine** (`127.0.0.1`). Ne remplacez pas ce réglage par `0.0.0.0` sans un proxy HTTPS devant.
- **Aucune clé d'API** n'est transmise aux assistants, et l'Atelier ne lit ni ne copie jamais leurs sessions.